جزوهٔ سیستم‌عامل — زمان‌بندی پردازنده

بر پایهٔ یادداشت‌های NotebookLM

درس: سیستم‌عامل

استاد / مدرس: دکتر نمونه

تهیه‌کننده: علی‌اکبر

تاریخ: 9 مهر 1405

این جزوه از یادداشت‌های NotebookLM تهیه و در قالب PDF بازآرایی شده است.

فهرست مطالب

فصل ۳ — زمان‌بندی پردازنده ۳

۱. مفاهیم پایه ۳

۲. الگوریتم‌های کلاسیک ۳

۲.۱ اولین‌آمده اولین‌خدمت‌گیرنده (FCFS) ۳

۲.۲ کوتاه‌ترین کار نخست (SJF) ۴

۲.۳ زمان‌بندی نوبت گردشی (Round Robin) ۴

۳. معیارهای ارزیابی و پیچیدگی ۴

۴. پیاده‌سازی سادهٔ یک صف آماده ۵

۵. جمع‌بندی ۵

فصل ۳ — زمان‌بندی پردازنده

در سیستم‌های چندبرنامه‌ای (Multiprogramming)، چند فرایند به‌طور هم‌زمان در حافظه حاضرند و پردازنده باید میان آن‌ها جابه‌جا شود. این جابه‌جایی توسط **زمان‌بند** (Scheduler) انجام می‌شود؛ مؤلفه‌ای از هستهٔ سیستم‌عامل که تصمیم می‌گیرد کدام فرایند در لحظهٔ بعد اجرا شود و چه مدت پردازنده در اختیارش باشد.

زمان‌بند با اتکا به صف آماده (Ready Queue) کار می‌کند. هر زمان که پردازنده آزاد شود یا فرایندی به صف آماده اضافه‌گردد، زمان‌بند باید یک تصمیم بگیرد؛ به همین دلیل الگوریتم انتخابی به‌شدت بر کارایی کل سیستم اثر می‌گذارد.

> **نکتهٔ مهم:** زمان‌بندی نه‌فقط یک بحث کارایی است، بلکه بر عدالت (Fairness) و زمان پاسخ (Response Time) کاربران نیز اثر مستقیم دارد. در سیستم‌های بلادرنگ، حتی چند میلی‌ثانیه تأخیر می‌تواند فاجعه‌بار باشد.

۱. مفاهیم پایه

پیش از بررسی الگوریتم‌ها باید چهار مفهوم را دقیق تعریف کنیم:

- **زمان رسیدن** (Arrival Time): لحظه‌ای که فرایند به صف آماده وارد می‌شود.
- **زمان اجرا** (Burst Time): مدت زمانی که فرایند برای تکمیل به پردازنده نیاز دارد.
- **زمان انتظار** (Waiting Time): مجموع زمانی که فرایند در صف آماده معطل مانده است.
- **زمان چرخش** (Turnaround Time): فاصلهٔ میان زمان رسیدن تا پایان اجرا؛ برابر است با زمان انتظار به‌علاوهٔ زمان اجرا.

خروجی زمان‌بند را می‌توان به‌صورت نمودار گانت (Gantt Chart) نمایش داد: محور افقی زمان و مستطیل‌هایی که بازهٔ اجرای هر فرایند را نشان می‌دهند.

۲. الگوریتم‌های کلاسیک

۲.۱ اولین‌آمده اولین‌خدمت‌گیرنده (FCFS)

ساده‌ترین سیاست ممکن: فرایندی که زودتر وارد صف آماده شده، زودتر اجرا می‌شود. این الگوریتم غیرقبضه‌ای (Non-preemptive) است؛ یعنی تا زمانی که فرایند جاری پایان نیابد، پردازنده آزاد نمی‌شود.

۱. فرایندها بر اساس زمان رسیدن مرتب می‌شوند.

۲. فرایند نخست تا پایان اجرا می‌کند.

۳. در پایان هر اجرا، فرایند بعدی از سر صف برداشته می‌شود.

مشکل اصلی FCFS، اثر کاروانی (Convoy Effect) است: اگر یک فرایند کند ابتدای صف قرار گیرد، فرایندهای کوچک پشت آن گرفتار می‌شوند و میانگین زمان انتظار به‌شدت بالا می‌رود.

۲.۲ کوتاه‌ترین کار نخست (SJF)

در این الگوریتم فرایندی که کمترین زمان اجرا را دارد، زودتر انتخاب می‌شود. SJF از نظر کمینه‌سازی میانگین زمان انتظار بهینه است، اما دو اشکال جدی دارد:

- زمان اجرای واقعی فرایندها معمولاً از پیش معلوم نیست و باید تخمین زده شود.
- فرایندهای طولانی ممکن است هرگز اجرا نشوند (پدیدهٔ قحطی یا Starvation).

۲.۳ زمان‌بندی نوبت گردشی (Round Robin)

هر فرایند یک برش زمانی (Time Quantum) کوتاه — مثلاً ۴ میلی‌ثانیه — دریافت می‌کند. اگر در پایان برش، اجرای فرایند تمام نشده باشد، از پردازنده گرفته و به انتهای صف فرستاده می‌شود. انتخاب اندازهٔ برش زمانی یک بده‌بستان است:

- برش خیلی کوچک ← سربار (Overhead) جابه‌جایی زمینه زیاد می‌شود.
- برش خیلی بزرگ ← الگوریتم به FCFS فروکاسته می‌شود.

جدول زیر مقایسهٔ سریع این سه الگوریتم را نشان می‌دهد.

الگوریتم	قبضه‌ای؟	بهینه از نظر میانگین انتظار	خطر قحطی	مناسب برای
FCFS	خیر	خیر	ندارد	کارهای دسته‌ای (Batch)
SJF	بله/خیر	بله	دارد	سیستم‌های دسته‌ای با تخمین دقیق
Round Robin	بله	خیر	ندارد	سیستم‌های اشتراک زمانی

۳. معیارهای ارزیابی و پیچیدگی

برای مقایسهٔ الگوریتم‌ها از معیارهایی مانند بهره‌وری پردازنده (CPU Utilization)، توان عملیاتی (Throughput) و زمان پاسخ استفاده می‌شود. هزینهٔ محاسباتی تصمیم زمان‌بند نیز مهم است؛ برای مثال مرتب‌سازی صف آماده با n فرایند از مرتبهٔ O(n log n) است و اگر پیچیدگی زمانی زمان‌بند از این مرتبه فراتر رود، خودِ زمان‌بندی به گلوگاه تبدیل می‌شود.

شکل کلی زمان انتظار در SJF را می‌توان به‌صورت زیر نوشت: فرمول میانگین انتظار برابر است با (∑ w_i) ÷ (n) که در آن w_i زمان انتظار فرایند i-ام و n تعداد فرایندها است. توجه کنید که ۲¹⁰ = ۱۰۲۴ و برای مقایسهٔ برش‌های زمانی، واحد ۲۰۲۱ میلی‌ثانیه یا ۲¹۰ میکروثانیه ممکن است به‌کار رود.

۴. پیاده‌سازی سادهٔ یک صف آماده

ساختار دادهٔ صف آماده معمولاً یک صف اولویت‌دار است. در زبان پایتون می‌توان SJF را در چند خط شبیه‌سازی کرد:

```python
import heapq

def sjf(processes):
    """processes: فهرست (نام، زمان رسیدن، زمان اجرا)"""
    time = 0
    ready, done = [], []
    processes.sort(key=lambda p: p[1])
    i = 0
    while i < len(processes) or ready:
        while i < len(processes) and processes[i][1] <= time:
            name, arrival, burst = processes[i]
            heapq.heappush(ready, (burst, arrival, name))
            i += 1
        if not ready:
            time = processes[i][1]
            continue
        burst, arrival, name = heapq.heappop(ready)
        time += burst
        done.append((name, time - arrival))
    return done
```

خروجی این تابع، زمان چرخش هر فرایند است و میانگین آن معیار مقایسه با سایر الگوریتم‌ها خواهد بود.

۵. جمع‌بندی

زمان‌بندی پردازنده نمونهٔ روشنی از یک تصمیم طراحی است که باید میان کارایی، سادگی و عدالت تعادل برقرار کند. هیچ الگوریتمی برای همهٔ بارهای کاری بهینه نیست؛ به همین دلیل هستهٔ سیستم‌عامل‌های امروزی از زمان‌بندهای چندسطحی با بازخورد (MLFQ) استفاده می‌کنند تا هر دو دستهٔ فرایندهای تعاملی و دسته‌ای را به‌طور معقول پوشش دهند.

منابع پیشنهادی: فصل ۵ کتاب «مفاهیم سیستم‌عامل» سیلبرشاتز و فصل ۷ کتاب OSTEP.